

PasswordStore Protocol Audit Report

Denzel Abelle

May 26, 2026

Contents

1 PasswordStore Audit Findings	1
1.1 High Severity	1
1.1.1 [H-1] Storing passwords in plaintext on the blockchain is not secure	1
1.1.2 [H-2] PasswordStore::setPassword lacks access control, allowing any user to change the stored password	2
1.2 Informational	2
1.2.1 [I-1] Incorrect Natspec documentation for PasswordStore::getPassword	2
1.2.2 [I-2] SetNewPassword event is missing the caller address	3

1 PasswordStore Audit Findings

1.1 High Severity

1.1.1 [H-1] Storing passwords in plaintext on the blockchain is not secure

Description: All data stored on the Ethereum blockchain is public and can be read by anyone, regardless of its visibility keyword (`private`, `internal`, etc.). The `PasswordStore::s_password` variable is intended to be a private secret, but since it is stored in contract storage, it can be accessed off-chain using tools like `eth_getStorageAt`.

Impact: The core security premise of the contract is broken. Anyone can retrieve the “private” password, leading to a complete loss of confidentiality.

Proof of Concept: The following steps demonstrate how an attacker can retrieve the password without calling any contract functions:

1. Deploy the contract.
2. Call `setPassword("mySecretPassword")`.
3. Use `cast` to read the storage slot 1 (where `s_password` is stored):

```
cast storage <CONTRACT_ADDRESS> 1 --rpc-url <RPC_URL>
```

4. Convert the hex result to a string:

```
cast parse-bytes32-string <HEX_RESULT>
```

Output: "mySecretPassword"

Recommended Mitigation: Sensitive data should never be stored in plaintext on-chain. Consider: 1.

Off-chain Encryption: Encrypt the password off-chain before storing it, and decrypt it off-chain after retrieval. 2. **Hashing:** If the goal is only to verify the password, store a hash (e.g., `keccak256`) instead of the plaintext. However, this would not allow the `getPassword` functionality to work as currently designed.

1.1.2 [H-2] PasswordStore::setPassword lacks access control, allowing any user to change the stored password

Description: The PasswordStore::setPassword function is marked as `external` but does not implement any access control checks (e.g., `require(msg.sender == s_owner)`). Although the Natspec documentation states that “This function allows only the owner to set a new password,” the implementation allows any address to call it.

```
function setPassword(string memory newPassword) external {
    s_password = newPassword; // @audit - No access control
    emit SetNewPassword();
}
```

Impact: Any malicious actor can overwrite the owner’s password, leading to a loss of data integrity and unauthorized modification of the contract state.

Proof of Concept: Add the following test to PasswordStore.t.sol to verify that a non-owner can change the password:

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.assume(randomAddress != owner);
    vm.prank(randomAddress);
    string memory expectedPassword = "hackedPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation: Add an access control check to the `setPassword` function to ensure only the owner can update the password.

```
function setPassword(string memory newPassword) external {
    if (msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }
    s_password = newPassword;
    emit SetNewPassword();
}
```

1.2 Informational

1.2.1 [I-1] Incorrect Natspec documentation for PasswordStore::getPassword

Description: The Natspec documentation for the `getPassword` function incorrectly references a parameter that does not exist.

```
/*
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set. // @audit - No such parameter
 */
function getPassword() external view returns (string memory) {
```

Impact: This leads to confusion for developers and auditors reading the code, as the documentation does not accurately describe the function’s signature.

Recommended Mitigation: Remove the `@param newPassword` line from the `getPassword` function's Natspec.

1.2.2 [I-2] SetNewPassword event is missing the caller address

Description: The `SetNewPassword` event is emitted whenever a new password is set, but it does not include the address of the user who performed the action.

```
event SetNewPassword();
```

Impact: It is difficult for off-chain monitoring tools to track who updated the password without analyzing the full transaction trace.

Recommended Mitigation: Update the event to include the address of the sender.

```
event SetNewPassword(address indexed user);
```